

```

import { useState, useEffect } from "react";

// =====
// DATA – Guide complet Claude AI (Production-ready, 2026)
// =====

const MODULES = [
  // _____
  // MODULE 0 – VUE D'ENSEMBLE
  // _____
  {
    id: "overview",
    icon: "◆",
    label: "Vue d'ensemble",
    color: "#E2E8F0",
    accent: "#94A3B8",
    chapters: [
      {
        title: "Pourquoi ce guide existe",
        content: `La plupart des gens utilisent Claude à 10% de son potentiel. Il

Les utilisateurs avancés, eux, construisent des  systèmes  : fichiers de config

Ce guide vous amène de l'installation à ce niveau – avec des fichiers prêts à l'e
      },
      {
        title: "Les 5 couches de Claude (à maîtriser dans l'ordre)",
        content: null,
        table: {
          headers: ["Couche", "Outil", "Ce que ça fait", "Quand le maîtriser"],
          rows: [
            ["1. Interface", "claude.ai / app", "Conversations, Projects, Mémoire", "2. Mémoire", "CLAUDE.md + Projects", "Contexte persistant chargé à", "3. Compétences", "Skills (.md)", "Procédures réutilisables auto-déc", "4. Connexions", "MCP (.mcp.json)", "Claude accède à vos outils réel", "5. Agents", "Subagents + Hooks", "Automatisation, parallélisation,
          ]
        }
      },
      {
        title: "Architecture globale de votre setup",
        content: null,
        code: {
          lang: "bash",

```

```

    label: "Structure de fichiers cible (~/.claude/ + projets)",
    body: `~/.claude/                                ← Configuration GLOBALE (tous proj

├─ CLAUDE.md                                          ← Instructions chargées à CHAQUE session
├─ CLAUDE.local.md                                  ← Vos overrides privés (gitignore)
├─ settings.json                                    ← Permissions globales de Claude Code
├─ skills/                                           ← Skills disponibles dans tous vos projets
│   ├─ code-review/SKILL.md
│   ├─ n8n-workflow/SKILL.md
│   ├─ site-web/SKILL.md
│   ├─ tunnel/SKILL.md
│   └─ linkedin-post/SKILL.md
├─ agents/                                           ← Subagents globaux
│   ├─ researcher.md
│   ├─ qa-tester.md
│   └─ deployer.md
├─ commands/                                        ← Slash commands personnelles /review, /deploy...
│   ├─ review.md
│   └─ deploy.md
└─ hooks/                                           ← Scripts de sécurité automatiques
    ├─ pre-commit.sh
    └─ pre-tool-use.sh

mon-projet/                                          ← Configuration PROJET (dans Git)
├─ CLAUDE.md                                          ← Règles spécifiques à ce projet
├─ .mcp.json                                          ← Serveurs MCP de ce projet
└─ .claude/
    ├─ rules/                                        ← Règles modulaires par domaine
    │   ├─ security.md
    │   └─ api-conventions.md
    ├─ skills/                                       ← Skills spécifiques à ce projet
    └─ agents/                                       ← Agents spécifiques à ce projet`
    }
  }
]
},

// _____
// MODULE 1 - INSTALLATION
// _____
{
  id: "install",
  icon: "⚡",
  label: "Installation",
  color: "#34D399",
  accent: "#059669",
  chapters: [
    {

```

```

    title: "1.1 – Prérequis système",
    content: null,
    table: {
      headers: ["Outil", "Version min.", "Vérification", "Installer si absent"],
      rows: [
        ["Node.js", "18.0+", "node --version", "nodejs.org/download"],
        ["npm", "9.0+", "npm --version", "Inclus avec Node.js"],
        ["Git", "2.30+", "git --version", "git-scm.com"],
        ["Terminal", "Bash/Zsh/PowerShell", "-", "WSL2 recommandé sur Windows"]
      ]
    }
  },
  {
    title: "1.2 – Installation Claude Code",
    content: null,
    code: {
      lang: "bash",
      label: "Installation et vérification complète",
      body: `# 1. Installation globale
npm install -g @anthropic-ai/claude-code

# 2. Vérifier la version
claude --version
# Attendu : claude-code/X.X.X ou similaire

# 3. Diagnostic complet (détecte les problèmes)
claude doctor

# 4. Premier lancement (authentification)
claude
# → Ouvre le navigateur pour vous connecter à votre compte Anthropic
# → Acceptez les permissions, revenez au terminal

# 5. Test rapide
cd /tmp && mkdir test-claude && cd test-claude
claude "Crée un fichier hello.txt avec 'Bonjour monde'"
# → Claude doit créer le fichier sans erreur`
    }
  },
  {
    title: "1.3 – Configuration Settings claude.ai",
    content: `**Checklist d'activation obligatoire** – allez dans Settings sur
    checklist: [
      "✅ Memory – Mémoire inter-conversations activée",
      "✅ Web Search – Recherche web temps réel",
      "✅ Code Execution & File Creation – Exécuter du code et créer des fichiers",
      "✅ Artifacts – Prévisualisation interactive des outputs",
    ]
  }
]

```

```

    "✅ Search and reference past chats – Retrouver des conversations",
    "✅ Generate memory from chat history – Mémorisation automatique",
    "⚙️ User Preferences – Rédiger vos préférences de ton, langue, format
  ]
},
{
  title: "1.4 – Votre User Preference (à copier-adapter)",
  content: null,
  code: {
    lang: "text",
    label: "À coller dans Settings > User Preferences",
    body: `Je m'appelle [Votre Prénom]. Je travaille depuis Hamburg, Allema
Langue : Français exclusivement sauf si je demande autrement.

```

Style de réponse :

- Structuré avec des titres clairs
- Exemples concrets et fichiers prêts à l'emploi
- Résumé actionnable en fin de réponse
- Pas de remplissage inutile

Mon niveau technique : avancé (APIs, scripts, CLI, Docker)

Stack principale : n8n, Claude API, JavaScript/Python, Cloudflare

Objectif : devenir expert Claude / Claude Certified Architect

Ne jamais :

- Inventer des données techniques sans les vérifier
- Donner des réponses génériques quand je pose une question précise
- M'expliquer ce que je connais déjà`

```

    }
  }
]
},

// -----
// MODULE 2 – CLAUDE.md & MÉMOIRE
// -----
{
  id: "claudemd",
  icon: "🧠",
  label: "CLAUDE.md & Mémoire",
  color: "#F59E0B",
  accent: "#D97706",
  chapters: [
    {
      title: "2.1 – Règles absolues pour CLAUDE.md",
      content: null,
      warning: `CRITIQUE – Ces règles évitent 90% des problèmes :

```

- Maximum 200 lignes. Au-delà, Claude « oublie » les règles du milieu.
- Chaque ligne compte comme une instruction. Budget total ≈ 100 règles utiles (Cl
- Jamais de documentation longue inline – référencer un fichier externe à la plac
- CLAUDE.local.md pour vos préférences privées (gitignore automatique).
- Les règles critiques (sécurité, permissions) doivent être dans les Hooks – CLAU

```

    },
    {
        title: "2.2 – Template CLAUDE.md global (production-ready)",
        content: null,
        code: {
            lang: "markdown",
            label: "~/ .claude/CLAUDE.md – À personnaliser et déployer maintenant",
            body: `# Mon Setup Claude – [Votre Nom]
# Dernière mise à jour : [date]

## Identité et contexte
- Basé à Hamburg, Allemagne
- Langue de travail : Français (anglais accepté pour le code)
- Niveau : Expert (APIs, scripts, automation, Claude ecosystem)
- Stack : n8n, Claude API, JavaScript, Python, Docker, Cloudflare

## Règles de comportement absolues
- Toujours répondre en français sauf si code ou variable technique
- Proposer un plan AVANT d'exécuter une tâche complexe
- Confirmer avant toute action destructive (delete, overwrite, deploy)
- Ne jamais committer sur main directement
- Alerter si une demande sort du périmètre prévu

## Stack et conventions
- Node.js : v20+, npm, ESM modules
- Python : 3.11+, pip, venv obligatoire
- Commits : format Conventional Commits (feat:, fix:, docs:, refactor:)
- Variables : camelCase JS, snake_case Python
- Secrets : toujours via variables d'environnement, jamais hardcodés

## Commandes fréquentes
# Node
npm run dev / npm test / npm run build / npm run lint

# Python
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Docker
docker compose up -d / docker compose logs -f / docker compose down

## Structure type de mes projets

```

Pour les détails d'architecture, lire le CLAUDE.md du projet concerné.

Ce que je fais régulièrement

- Tunnels Cloudflare pour exposer des services locaux
- Sites web (landing pages, dashboards, apps)
- Workflows n8n connectés à Claude API
- Automatisations multi-plateformes (Telegram, email, Slack)
- CVs/documents pour contacts en Allemagne

Projets actifs

- ~/projets/n8n-automation/ : workflows automation
- ~/projets/api-integrations/ : intégrations Claude API
- ~/projets/sites/ : sites web clients`

}

},

{

```
  title: "2.3 – Template CLAUDE.md par type de projet",
  content: "Créer un CLAUDE.md dans chaque projet. Voici les templates par
code: {
  lang: "markdown",
  label: "Template projet SITE WEB (à placer dans mon-site/CLAUDE.md)",
  body: `# Projet : [Nom du site]
```

Type : Site web / Landing page / Dashboard

Stack

- Framework : [Next.js / Astro / HTML vanilla / React]
- CSS : [Tailwind / CSS modules / Styled Components]
- Hébergement cible : [Vercel / Cloudflare Pages / VPS]
- Domaine : [domaine.com]

Commandes

```
npm run dev          # Serveur local sur port 3000
npm run build        # Build production
npm run preview      # Préview du build
npm run deploy       # Déploiement (si configuré)
```

Structure

src/

```
├─ pages/           # Pages (ou app/ pour Next.js App Router)
├─ components/      # Composants réutilisables
├─ styles/          # CSS global
└─ assets/          # Images, fonts
```

Conventions

- Composants : PascalCase (HeroSection.jsx)
- Pas d'images > 200KB sans compression
- Toujours mobile-first

- Dark mode via CSS variables

Déploiement

Lire docs/deployment.md avant tout déploiement en production.`

```
    }
  },
  {
    title: "2.4 – Template CLAUDE.md pour workflows n8n",
    content: null,
    code: {
      lang: "markdown",
      label: "Template projet WORKFLOW N8N (mon-workflow/CLAUDE.md)",
      body: `# Projet : [Nom du workflow]
# Type : Automatisation n8n
```

Contexte

- Instance n8n : [http://localhost:5678 / URL cloud]
- Webhook base URL : [URL publique via Cloudflare Tunnel ou ngrok]
- Claude API : Sonnet 4.6 (production), Haiku 4.5 (tests)

Architecture du workflow

[Décrire la logique en 3-5 lignes]

Ex: Webhook reçoit email → Claude analyse → Si urgent → Telegram alert

Nœuds n8n utilisés

- Webhook trigger
- HTTP Request (Claude API)
- Set / Code node
- Telegram / Gmail / Slack

Appel Claude API dans n8n

URL: https://api.anthropic.com/v1/messages

Headers: x-api-key, anthropic-version: 2023-06-01

Body model: claude-sonnet-4-6

Règles

- Toujours tester avec des données mockées avant d'activer
- Logger les erreurs dans un workflow dédié
- Cache_control sur les system prompts répétitifs (économie 90%)
- Limiter les boucles : max 50 itérations par exécution

Credentials n8n

Ne jamais committer les credentials. Stocker dans n8n Credentials UI.`

```
    }
  }
]
},
```

```
// -----
// MODULE 3 – PROJECTS & PROMPTS
// -----
{
  id: "projects",
  icon: "🎯",
  label: "Projects & Prompts",
  color: "#60A5FA",
  accent: "#2563EB",
  chapters: [
    {
      title: "3.1 – Créer un Project efficace sur claude.ai",
      content: `Un Project = contexte permanent + documents de référence + inst
Règle : créer un Project pour chaque domaine de travail récurrent.` ,
      table: {
        headers: ["Project à créer", "Instructions clés à inclure", "Documents
        rows: [
          ["Dev / Code", "Stack, conventions, règles de sécurité", "Architectur
          ["Sites web clients", "Charte graphique, ton, SEO rules", "Brief clie
          ["Workflows n8n", "Structure JSON n8n, conventions nœuds", "Workflows
          ["Contacts Allemagne", "Règles Lebenslauf, ATS, format allemand", "Ex
          ["Contenu / LinkedIn", "Ton de marque, thèmes, longueur posts", "Post
        ]
      }
    },
    {
      title: "3.2 – Template d'instructions Project (copier-adapter)",
      content: null,
      code: {
        lang: "text",
        label: "Instructions système d'un Project (dans le champ 'Instructions'
        body: `# Rôle
Tu es mon assistant expert en [DOMAINE], opérant dans le contexte de [NOM DU PROJ

# Contexte permanent
- Entreprise/Projet : [Nom]
- Secteur : [Domaine]
- Localisation : Hamburg, Allemagne
- Stack technique : [Liste des outils]

# Style de réponse
- Langue : Français exclusivement
- Format : Structuré avec titres, code formaté, tables quand pertinent
- Longueur : Ajustée à la complexité (pas de padding inutile)
- Toujours terminer par : une checklist d'actions concrètes
```



```

# Règles métier
[Lister 5-10 règles spécifiques à ce domaine]
Ex:
- Toujours proposer 3 variantes avant de trancher
- Vérifier la compatibilité mobile pour tout output visuel
- Respecter les standards WCAG 2.1 AA

# Ce qu'il ne faut jamais faire
- Ne jamais inventer des données chiffrées
- Ne jamais proposer de solutions qui nécessitent des outils non listés
- Ne pas répéter ce que je viens de dire dans le prompt

# Documents de référence
Les fichiers uploadés dans ce Project font autorité. Les consulter avant de répon
    }
  },
  {
    title: "3.3 – Structure de prompt expert (adaptable à tout projet)",
    content: null,
    code: {
      lang: "text",
      label: "Template prompt universel – à adapter selon le contexte",
      body: `<contexte>
[Situation actuelle, ce qui existe déjà, contraintes connues]
Ex: "Je construis un tunnel Cloudflare pour exposer mon n8n local.
      n8n tourne sur port 5678. J'ai un compte Cloudflare gratuit.
      OS : Ubuntu 22.04 sur VPS Hetzner 2Go RAM."
</contexte>

<objectif>
[Ce que je veux obtenir précisément – output attendu]
Ex: "Obtenir une URL publique stable type
      https://mon-n8n.mondomaine.com pointant vers localhost:5678,
      avec HTTPS automatique et redémarrage automatique."
</objectif>

<contraintes>
[Ce qui ne peut pas changer, limites techniques, budget]
Ex: "Gratuit ou quasi-gratuit. Pas de serveur supplémentaire.
      Doit survivre à un reboot du VPS."
</contraintes>

<format_attendu>
[Comment je veux la réponse]
Ex: "Commandes bash prêtes à copier-coller, dans l'ordre d'exécution,
      avec commentaires explicatifs. Terminer par un checklist de vérification."
</format_attendu>`

```

```

    }
  },
  {
    title: "3.4 – Prompts spécialisés par type de projet",
    content: null,
    code: {
      lang: "text",
      label: "Prompt TUNNEL CLOUDFLARE – Copier-coller direct",
      body: `<contexte>
Je veux exposer [SERVICE] tournant sur [PORT] en local via Cloudflare Tunnel.
OS : [Ubuntu 22.04 / Windows WSL2 / macOS]
Domaine : [mondomaine.com] (déjà sur Cloudflare / pas encore)
Service : [n8n / site web / API / autre]
</contexte>

<objectif>
URL publique sécurisée HTTPS : https://[sous-domaine].[domaine].com
Tunnel persistant (survit aux reboots)
Redémarre automatiquement si crash
</objectif>

<format_attendu>
1. Commandes d'installation cloudflared
2. Commande d'authentification Cloudflare
3. Commande de création du tunnel
4. Configuration DNS (ce que je dois faire dans Cloudflare dashboard)
5. Configuration service systemd pour démarrage automatique
6. Checklist de vérification finale
</format_attendu>`
    }
  },
  {
    title: "3.5 – Prompt SITE WEB production",
    content: null,
    code: {
      lang: "text",
      label: "Prompt SITE WEB – Structure complète",
      body: `<contexte>
Type de site : [landing page / portfolio / dashboard / e-commerce / blog]
Audience : [qui va l'utiliser]
Objectif principal : [ce que le visiteur doit faire]
Stack souhaitée : [HTML/CSS vanilla / React / Next.js / Astro]
Hébergement : [Vercel / Cloudflare Pages / VPS / GitHub Pages]
</contexte>

<objectif>
Site complet avec :

```

```
- Page(s) : [liste des pages nécessaires]
- Design : [ton / style / références visuelles]
- Fonctionnalités : [formulaire de contact / blog / auth / etc]
- SEO : [meta tags, Open Graph, sitemap]
- Performance : [score Lighthouse cible > 90]
</objectif>
```

```
<contraintes>
```

```
- Mobile-first obligatoire
- Chargement < 2s
- Accessibilité WCAG 2.1 AA
- [Autres contraintes spécifiques]
```

```
</contraintes>
```

```
<format_attendu>
```

```
1. Structure des fichiers du projet
2. Code complet de chaque fichier (prêt à déployer)
3. Commandes d'installation et de déploiement
4. Checklist avant mise en ligne
```

```
</format_attendu>`
```

```
    }
  }
]
},
```

```
// _____
// MODULE 4 – SKILLS
// _____
```

```
{
  id: "skills",
  icon: "⚙️",
  label: "Skills",
  color: "#A78BFA",
  accent: "#7C3AED",
  chapters: [
    {
      title: "4.1 – Comprendre les Skills (différence commandes / skills)",
      content: null,
      table: {
        headers: ["Type", "Déclencheur", "Usage", "Emplacement"],
        rows: [
          ["Slash Command", "Vous tapez /nom-commande", "Workflow explicite à l'usage de Claude", "Vous pouvez utiliser des commandes slash pour déclencher des actions spécifiques à Claude. Par exemple, /help pour obtenir de l'aide, /clear pour effacer le contexte, ou /remember pour sauvegarder des informations."],
          ["Skill", "Claude reconnaît le contexte automatiquement", "Procédure automatisée", "Une fois configurée, une compétence (skill) permet à Claude d'effectuer une tâche spécifique sans que vous n'ayez à lui donner des instructions détaillées à chaque fois. Par exemple, une skill de recherche peut automatiquement trouver des informations à jour sur Internet."],
          ["Agent", "Tâche déléguée avec contexte isolé", "Recherche, review, Q&A", "Vous pouvez déléguer une tâche complexe à un agent autonome. L'agent agit dans un contexte isolé, peut utiliser des outils (comme la recherche ou l'accès à des bases de données), et vous fournit un rapport détaillé de ses actions et de ses conclusions."],
        ]
      }
    }
  ],
},
```

```

    {
      title: "4.2 – Skill : Revue de code (prêt à déployer)",
      content: null,
      code: {
        lang: "markdown",
        label: "~/claude/skills/code-review/SKILL.md",
        body: `---
name: code-review
description: Revue de code complète. S'active quand on parle de PR,
              merge, review de code, qualité de code, ou avant déploiement.
---

# Procédure de revue de code

## Déclencheurs
Activer ce Skill quand : review de code, "regarde mon code",
"c'est bon pour merger ?", "avant de déployer", "audit du code".

## Étapes systématiques (dans l'ordre)
1. Lire l'INTÉGRALITÉ du code avant tout commentaire
2. Vérifier la sécurité (injections, secrets exposés, CORS)
3. Vérifier la logique métier (cas limites, erreurs silencieuses)
4. Évaluer la lisibilité (nommage, complexité, commentaires)
5. Identifier les dépendances inutiles ou vulnérables
6. Proposer des optimisations de performance si pertinent

## Format de sortie OBLIGATOIRE

### ✅ Points forts
[Ce qui est bien fait – soyez précis]

### ⚠️ À améliorer (non-bloquant)
[Problèmes mineurs + suggestion concrète]

### 🛑 Bloquants (à corriger avant merge/déploiement)
[Problèmes critiques numérotés avec explication + correction]

### 📝 Code corrigé
[Extrait du code corrigé pour les points bloquants uniquement]

### ✔️ Verdict
APPROUVÉ / APPROUVÉ avec réserves / REFUSÉ (raison en 1 ligne)`
      },
    },
    {
      title: "4.3 – Skill : Création workflow n8n",
      content: null,

```

```

        code: {
            lang: "markdown",
            label: "~/claude/skills/n8n-workflow/SKILL.md",
            body: `---
name: n8n-workflow
description: Conception et implémentation de workflows n8n.
            S'active pour tout ce qui concerne n8n, automation,
            workflows, intégrations entre services.
            ---

# Procédure création workflow n8n

## Avant de commencer
1. Demander si l'instance n8n est locale ou cloud
2. Demander l'URL publique disponible (nécessaire pour webhooks)
3. Identifier les credentials déjà configurés dans n8n

## Phase 1 – Architecture (toujours faire avant le code)
- Schéma textuel : Trigger → Nœud 1 → Condition → Nœud 2 → Output
- Lister les APIs/services impliqués
- Identifier les points de failure possibles
- Confirmer avec l'utilisateur avant de continuer

## Phase 2 – Implémentation
Produire le JSON n8n complet importable directement.

Structure JSON n8n obligatoire :
\`\`\`json
{
    "name": "Nom du workflow",
    "nodes": [...],
    "connections": {...},
    "settings": {
        "errorWorkflow": "",
        "timezone": "Europe/Berlin"
    },
    "tags": []
}
\`\`\`

## Intégration Claude API dans n8n
Nœud : HTTP Request
- URL: https://api.anthropic.com/v1/messages
- Method: POST
- Headers: x-api-key (depuis Credential), anthropic-version: 2023-06-01
- Body: { "model": "claude-sonnet-4-6", "max_tokens": 1024, "messages": [...] }
- Extract: {{ $json.content[0].text }}

```

```

## Optimisation coûts
Si le system prompt est identique sur toutes les requêtes :
Ajouter cache_control: {"type": "ephemeral"} → économie jusqu'à 90%

## Output final
1. JSON du workflow (importable dans n8n)
2. Variables d'environnement nécessaires
3. Credentials à configurer dans n8n
4. Instructions de test pas-à-pas
5. Scénarios d'erreur et comment les gérer`
    }
  },
  {
    title: "4.4 – Skill : Tunnel Cloudflare",
    content: null,
    code: {
      lang: "markdown",
      label: "~/ .claude/skills/tunnel/SKILL.md",
      body: `---
name: tunnel
description: Configuration de tunnels pour exposer des services locaux.
              S'active pour : tunnel, exposer localement, URL publique,
              cloudflared, ngrok, accès externe.
---

# Procédure configuration tunnel

## Questions à poser en premier
1. OS de la machine hôte ? (Ubuntu / Debian / Windows / macOS)
2. Service à exposer et port ? (n8n:5678, site:3000, API:8000...)
3. Domaine disponible sur Cloudflare ? (oui/non)
4. Besoin de persistance (tunnel survive au reboot) ?

## Option A – Cloudflare Tunnel (recommandé, gratuit avec domaine)

### Installation cloudflared
\\`\\`\\`bash
# Ubuntu/Debian
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudf
sudo dpkg -i cloudflared.deb

# Vérification
cloudflared --version
\\`\\`\\`

### Authentification et création tunnel

```

```

\\`bash
# Authentification (ouvre le navigateur)
cloudflared tunnel login

# Créer le tunnel (garder l'UUID généré)
cloudflared tunnel create mon-tunnel

# Créer la config
mkdir -p ~/.cloudflared
cat > ~/.cloudflared/config.yml << EOF
tunnel: <UUID-DU-TUNNEL>
credentials-file: /root/.cloudflared/<UUID>.json

ingress:
  - hostname: service.mondomaine.com
    service: http://localhost:PORT
  - service: http_status:404
EOF

# Créer le DNS CNAME
cloudflared tunnel route dns mon-tunnel service.mondomaine.com

# Test manuel
cloudflared tunnel run mon-tunnel
\\`

### Service systemd (démarrage automatique)
\\`bash
sudo cloudflared service install
sudo systemctl enable cloudflared
sudo systemctl start cloudflared
sudo systemctl status cloudflared
\\`

## Option B – ngrok (rapide, sans domaine, gratuit limité)
\\`bash
# Installation
curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trustedkeyrings/ngrok.asc &&
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list &&
sudo apt update && sudo apt install ngrok

# Authentification (compte ngrok.com requis)
ngrok config add-authtoken VOTRE_TOKEN

# Exposer un service
ngrok http PORT

```

```

# URL permanente (plan payant) ou utiliser Cloudflare à la place
\\`

## Checklist de vérification finale
- [ ] cloudflared tunnel list → tunnel visible
- [ ] curl https://service.mondomaine.com → réponse 200
- [ ] Reboot du serveur + vérification que le tunnel redémarre
- [ ] HTTPS valide (pas d'avertissement SSL)
- [ ] Logs sans erreurs : journalctl -u cloudflared -f`
    }
  },
  {
    title: "4.5 – Skill : Création site web",
    content: null,
    code: {
      lang: "markdown",
      label: "~/claude/skills/site-web/SKILL.md",
      body: `---
name: site-web
description: Création de sites web complets. S'active pour :
      créer un site, landing page, portfolio, dashboard web,
      page web, HTML/CSS, React, Next.js.
---

# Procédure création site web

## Phase 0 – Brief (OBLIGATOIRE avant toute ligne de code)
Collecter ces informations si non fournies :
- Type de site (landing / portfolio / app / e-commerce / blog)
- Audience cible (qui va l'utiliser)
- Action principale (CTA : s'inscrire / contacter / acheter / lire)
- Stack préférée ou contrainte
- Hébergement cible
- Délai et niveau de finition attendu

## Phase 1 – Choix de stack
\\`
Site vitrine simple      → HTML/CSS/JS vanilla + Cloudflare Pages
Landing page marketing   → Astro ou HTML vanilla
App interactive          → React + Vite ou Next.js
Dashboard                → React + Recharts/shadcn
Blog/contenu             → Astro ou Next.js
\\`

## Phase 2 – Structure fichiers (proposer avant de coder)
Toujours présenter l'arborescence et la faire valider.

```



```
## Phase 3 – Standards de développement
- Mobile-first (breakpoints: 375px, 768px, 1024px, 1440px)
- CSS variables pour les couleurs/espacements
- Images : WebP, lazy loading, taille maximale 200KB
- Performance : viser Lighthouse > 90 sur les 4 métriques
- SEO : title, description, Open Graph sur chaque page
- Accessibilité : alt sur toutes les images, aria-labels sur les boutons
```

```
## Phase 4 – Déploiement
```

```
### Cloudflare Pages (recommandé, gratuit)
```

```
\`\`\`bash
```

```
# Via CLI
```

```
npm install -g wrangler
```

```
wrangler pages project create mon-site
```

```
wrangler pages deploy ./dist --project-name=mon-site
```

```
# Via Git (auto-deploy)
```

```
# Connecter le repo GitHub dans Cloudflare Dashboard > Pages
```

```
\`\`\`
```

```
### Vercel
```

```
\`\`\`bash
```

```
npm install -g vercel
```

```
vercel --prod
```

```
\`\`\`
```

```
## Phase 5 – Checklist avant livraison
```

- [] Build production sans erreurs (npm run build)
- [] Test sur mobile (Chrome DevTools, iPhone SE et Pixel 5)
- [] Lighthouse > 90 sur les 4 métriques
- [] Tous les liens fonctionnent
- [] Formulaires testés (si présents)
- [] Favicon configuré
- [] Meta description < 160 caractères
- [] HTTPS activé sur le domaine`

```
    }
```

```
  },
```

```
{
```

```
  title: "4.6 – Installer les Skills en une commande",
```

```
  content: null,
```

```
  code: {
```

```
    lang: "bash",
```

```
    label: "Script de setup automatique de tous les Skills",
```

```
    body: `#!/bin/bash
```

```
# setup-skills.sh – Crée la structure de Skills complète
```

```
mkdir -p ~/.claude/skills/code-review
```

```
mkdir -p ~/.claude/skills/n8n-workflow
mkdir -p ~/.claude/skills/tunnel
mkdir -p ~/.claude/skills/site-web
mkdir -p ~/.claude/skills/linkedin-post
mkdir -p ~/.claude/agents
mkdir -p ~/.claude/commands
mkdir -p ~/.claude/hooks
```

```
echo "✅ Structure Skills créée dans ~/.claude/"
echo "→ Maintenant copier le contenu de chaque SKILL.md depuis ce guide"
echo "→ Puis relancer : claude doctor"
```

```
    }
  }
]
},
```

```
// _____
// MODULE 5 – MCP
// _____
```

```
{
  id: "mcp",
  icon: "🔌",
  label: "MCP (Connecteurs)",
  color: "#22D3EE",
  accent: "#0891B2",
  chapters: [
    {
      title: "5.1 – Architecture MCP expliquée",
      content: `MCP = Model Context Protocol. Le standard universel qui donne à
```

Sans MCP : Claude lit des fichiers et exécute des commandes bash.

Avec MCP : Claude accède à GitHub, votre base de données, Slack, n8n, Telegram, e

```
table: {
  headers: ["Transport", "Description", "Usage"],
  rows: [
    ["stdio (local)", "Processus local lancé par Claude Code", "GitHub, f",
    ["HTTP/SSE (distant)", "Serveur web distant, appel HTTP", "APIs cloud",
    ["Desktop Extension (.dxt)", "Installation 1 clic dans claude.ai", "D
  ]
}
},
{
  title: "5.2 – .mcp.json de référence (production-ready)",
  content: null,
  code: {
    lang: "json",
    label: ".mcp.json – Placer à la racine de chaque projet (NE PAS committ
```

```

        body: `{
"mcpServers": {

  "github": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-github"],
    "env": {
      "GITHUB_PERSONAL_ACCESS_TOKEN": "\${GITHUB_TOKEN}"
    }
  },

  "filesystem": {
    "command": "npx",
    "args": [
      "-y", "@modelcontextprotocol/server-filesystem",
      "/home/user/projects",
      "/home/user/documents"
    ],
    "env": {}
  },

  "brave-search": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-brave-search"],
    "env": {
      "BRAVE_API_KEY": "\${BRAVE_API_KEY}"
    }
  },

  "sqlite": {
    "command": "npx",
    "args": [
      "-y", "@modelcontextprotocol/server-sqlite",
      "--db-path", "./data/app.db"
    ],
    "env": {}
  },

  "n8n": {
    "command": "npx",
    "args": ["-y", "n8n-mcp"],
    "env": {
      "N8N_HOST": "http://localhost:5678",
      "N8N_API_KEY": "\${N8N_API_KEY}"
    }
  }
}

```

```

    }
  }`
  },
  {
    title: "5.3 – Variables d'environnement (JAMAIS dans .mcp.json)",
    content: null,
    code: {
      lang: "bash",
      label: "~/.bashrc ou ~/.zshrc – Ajouter ces lignes",
      body: `# Tokens MCP (NE JAMAIS mettre les vraies valeurs dans .mcp.json
export GITHUB_TOKEN="ghp_votre_token_ici"
export BRAVE_API_KEY="votre_brave_api_key"
export N8N_API_KEY="votre_n8n_api_key"
export ANTHROPIC_API_KEY="sk-ant-votre_clé"
export CLOUDFLARE_API_TOKEN="votre_token_cloudflare"

# Rechargement
source ~/.bashrc # ou source ~/.zshrc`
    }
  },
  {
    title: "5.4 – Ordre d'installation des MCP (par priorité)",
    content: null,
    table: {
      headers: ["Priorité", "MCP", "Commande d'install", "Débloque"],
      rows: [
        ["1 – JOUR 1", "filesystem", "npx @modelcontextprotocol/server-filesy",
        ["2 – JOUR 1", "GitHub", "npx @modelcontextprotocol/server-github", "
        ["3 – SEMAINE 1", "Brave Search", "npx @modelcontextprotocol/server-b
        ["4 – SEMAINE 1", "SQLite/Postgres", "npx @modelcontextprotocol/serve
        ["5 – SEMAINE 2", "n8n MCP", "npx n8n-mcp", "Créer/modifier des workf
        ["6 – SEMAINE 2", "Google Workspace", "npm install -g @googleworkspac
        ["7 – OPTIONNEL", "Telegram", "npx @modelcontextprotocol/server-teleg
      ]
    }
  },
  {
    title: "5.5 – Commandes de diagnostic MCP",
    content: null,
    code: {
      lang: "bash",
      label: "Déboguer un MCP qui ne fonctionne pas",
      body: `# 1. Lister tous les serveurs MCP enregistrés
claude mcp list

# 2. Tester un serveur spécifique

```

```
claude mcp test github
claude mcp test filesystem
```

```
# 3. Voir les outils disponibles d'un serveur
claude mcp list-tools github
```

```
# 4. Logs en temps réel
tail -f ~/.claude/logs/mcp.log
```

```
# 5. Vérifier les variables d'environnement
echo $GITHUB_TOKEN      # Doit afficher le token (pas vide)
echo $N8N_API_KEY
```

```
# 6. Réinstaller un serveur problématique
npx -y @modelcontextprotocol/server-github --version
```

```
# Problème fréquent : variables d'env non disponibles pour Claude Code
# Solution : les exporter dans ~/.bashrc ET les vérifier avec "claude doctor"
    }
  },
  {
    title: "5.6 – MCP n8n : Claude construit vos workflows automatiquement",
    content: `Avec le MCP n8n connecté, Claude Code peut créer, modifier et d
code: {
  lang: "bash",
  label: "Exemples de prompts après connexion MCP n8n",
  body: `# Créer un workflow complet depuis Claude Code :
"Crée un workflow n8n qui : reçoit un webhook,
envoie le body à Claude Sonnet pour analyse,
si Claude répond 'urgent' → envoie un message Telegram,
sinon → logue dans un Google Sheet"
```

```
# Corriger un workflow existant :
"Le workflow 'Email digest' plante sur le nœud 3.
Lis-le, identifie le problème, corrige-le et redéploie."
```

```
# Optimiser les coûts :
"Ajoute du prompt caching sur tous les workflows
qui utilisent Claude API avec un system prompt fixe"
    }
  }
]
},
```

```
// _____
// MODULE 6 – HOOKS & SÉCURITÉ
// _____
```

```

{
  id: "hooks",
  icon: "🛡️",
  label: "Hooks & Sécurité",
  color: "#F87171",
  accent: "#DC2626",
  chapters: [
    {
      title: "6.1 – Pourquoi les Hooks sont CRITIQUES",
      content: null,
      warning: `CLAUDE.md a ~70% de fiabilité sur les règles critiques.
Les Hooks ont 100% de fiabilité – ils exécutent du code réel.

Pour les règles qui concernent la sécurité, les données de production, ou les act
    },
    {
      title: "6.2 – Hook de sécurité principal (déployer immédiatement)",
      content: null,
      code: {
        lang: "bash",
        label: "~/claude/hooks/pre-tool-use.sh – Exécuté avant CHAQUE action d
        body: `#!/bin/bash

# Hook de sécurité – pré-action Claude Code
# Variables disponibles : $CLAUDE_TOOL_NAME, $CLAUDE_TOOL_INPUT

TOOL="$CLAUDE_TOOL_NAME"
INPUT="$CLAUDE_TOOL_INPUT"
LOGFILE="$HOME/.claude/audit.log"

# — Log de toutes les actions —
echo "$(date '+%Y-%m-%d %H:%M:%S') | $TOOL | $INPUT" >> "$LOGFILE"

# — Bloquer accès aux fichiers sensibles —
if echo "$INPUT" | grep -qiE '\\.env$|\\.key$|\\.pem$|credentials\\.json|secrets\\
  echo "🚫 BLOQUÉ : Tentative d'accès à un fichier sensible"
  echo "Fichier détecté dans : $INPUT"
  exit 1
fi

# — Bloquer les commandes destructives sans confirmation —
if echo "$INPUT" | grep -qE 'rm -rf|DROP TABLE|DELETE FROM.*WHERE 1|format C:|shu
  echo "🚫 BLOQUÉ : Commande destructive détectée"
  echo "Commande : $INPUT"
  echo "Exécuter manuellement si intentionnel."
  exit 1
fi

```

```

# — Bloquer push direct sur main —
if echo "$INPUT" | grep -qE 'git push.*main|git push.*master'; then
    echo "🚫 BLOQUÉ : Push direct sur main interdit"
    echo "Créer une branch et une PR à la place."
    exit 1
fi

exit 0 # Autoriser toutes les autres actions`
    }
},
{
    title: "6.3 – Hook pre-commit",
    content: null,
    code: {
        lang: "bash",
        label: "~/.claude/hooks/pre-commit.sh – Avant chaque commit Git",
        body: `#!/bin/bash

# Vérifications avant commit

# Bloquer les fichiers sensibles dans le staging
STAGED=$(git diff --cached --name-only 2>/dev/null)

if echo "$STAGED" | grep -qE '\\.env$|\\.env\\.|credentials|secrets|\\.pem|\\.k
    echo "🚫 BLOQUÉ : Fichiers sensibles dans le commit"
    echo "Fichiers détectés : $(echo "$STAGED" | grep -E '\\.env|credentials|secret
    echo "Ajouter ces fichiers à .gitignore puis 'git rm --cached <fichier>'"
    exit 1
fi

# Vérifier que les tests passent (si npm test existe)
if [ -f "package.json" ] && grep -q '"test"' package.json; then
    echo "🔍 Exécution des tests avant commit..."
    npm test --silent
    if [ $? -ne 0 ]; then
        echo "🚫 BLOQUÉ : Les tests échouent. Corriger avant de committer."
        exit 1
    fi
fi

echo "✅ Pre-commit OK"
exit 0`
    }
},
{
    title: "6.4 – settings.json (permissions Claude Code)",
    content: null,
    code: {

```

```

        lang: "json",
        label: "~/ .claude/settings.json - Permissions globales",
        body: `{
"permissions": {
  "allow": [
    "Bash(git:*)",
    "Bash(npm:*)",
    "Bash(python:*)",
    "Bash(docker:*)",
    "Bash(cloudflared:*)",
    "Read(*)",
    "Write(~/.projets/*)",
    "Write(/tmp/*)"
  ],
  "deny": [
    "Bash(sudo rm -rf *)",
    "Bash(dd if=*)",
    "Write(/etc/*)",
    "Write(/usr/*)",
    "Write(~/.ssh/*)"
  ]
},
"hooks": {
  "PreToolUse": [
    {
      "matcher": "*",
      "hooks": [{"type": "command", "command": "~/ .claude/hooks/pre-tool-use.sh"}]
    }
  ]
}
}`
    }
  ]
},

```

```

// _____
// MODULE 7 - AGENTS & ORCHESTRATION
// _____

```

```

{
  id: "agents",
  icon: "🤖",
  label: "Agents & Orchestration",
  color: "#FB923C",
  accent: "#EA580C",
  chapters: [
    {

```



```

    title: "7.1 – Quand utiliser un Subagent",
    content: null,
    table: {
      headers: ["Situation", "Solution", "Pourquoi"],
      rows: [
        ["Tâche de recherche longue (50+ sources)", "Subagent researcher", "I"],
        ["Revue de code indépendante", "Subagent code-reviewer", "Regard neuf"],
        ["Tests QA d'un output", "Subagent qa-tester", "Valide sans biais de"],
        ["Déploiement prod (actions risquées)", "Subagent deployer avec permi"],
        ["Travail en parallèle (3 tâches simultanées)", "3 Subagents lancés e"]
      ]
    }
  },
  {
    title: "7.2 – Template Subagent universel",
    content: null,
    code: {
      lang: "markdown",
      label: "~/ .claude/agents/researcher.md",
      body: `---
name: researcher
description: Agent de recherche documentaire. Utiliser pour tout ce
      qui nécessite de chercher, synthétiser, compiler des informations
      sans modifier de fichiers.
model: claude-sonnet-4-6
tools:
  - web_search
  - read_file
# PAS d'accès write – lecture seule
---

# Agent Researcher

## Rôle
Je suis un agent de recherche spécialisé. Mon seul rôle :
trouver, vérifier et synthétiser des informations.

## Règles absolues
- Je ne modifie JAMAIS de fichiers
- Je cite systématiquement mes sources
- Je signale l'âge de l'information (date de publication)
- Je distingue "vérifié" de "probable" de "incertain"
- Je résume en maximum 500 mots sauf demande contraire

## Format de sortie
### 📄 Résumé (3-5 lignes)
[Ce que j'ai trouvé en résumé]

```

```

### 📖 Sources (avec dates)
- [Source 1] - [date] - [URL si disponible]
- [Source 2] - [date] - [URL si disponible]

### ⚠️ Incertitudes
[Ce que je n'ai pas pu vérifier ou ce qui semble contradictoire]`
    }
  },
  {
    title: "7.3 – Intégration Claude API + n8n (stack complète)",
    content: null,
    code: {
      lang: "json",
      label: "Configuration nœud HTTP Request n8n → Claude API",
      body: `// Configuration du nœud HTTP Request dans n8n
// Paramètres :

URL: https://api.anthropic.com/v1/messages
Method: POST

Headers:
{
  "x-api-key": "{{ $credentials.anthropicApiKey }}",
  "anthropic-version": "2023-06-01",
  "content-type": "application/json"
}

Body (JSON) – Version AVEC cache (économie 90% sur system prompt) :
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": [
    {
      "type": "text",
      "text": "Votre system prompt ici – identique sur toutes les requêtes",
      "cache_control": {"type": "ephemeral"}
    }
  ],
  "messages": [
    {
      "role": "user",
      "content": "{{ $json.userMessage }}"
    }
  ]
}

```

```
// Extraire la réponse dans le nœud suivant :
{{ $json.content[0].text }}`
    }
  },
  {
    title: "7.4 – Workflow auto-réparant (Claude + n8n avancé)",
    content: `Pattern avancé : n8n détecte une erreur → alerte Telegram → app
code: {
  lang: "text",
  label: "Architecture du workflow auto-réparant",
  body: `WORKFLOW : Auto-repair n8n`
}
```

1. ERROR TRIGGER

└ Capture toutes les erreurs de workflows

2. SET NODE – Formatter l'erreur

└ Préparer : workflowName, nodeFailed, errorMessage, timestamp

3. HTTP REQUEST → Claude API

└ Prompt: "Voici l'erreur du workflow [name] sur le nœud [node] :
[errorMessage]
Propose une correction JSON pour ce nœud."

4. TELEGRAM → Envoyer à l'admin

└ Message + boutons : [ Appliquer la correction] [ Ignorer]

5. WEBHOOK (sur approbation)

└ Reçoit la réponse de l'admin

6. HTTP REQUEST → API n8n

└ PATCH /api/v1/workflows/{id}
Body: correction proposée par Claude

7. TELEGRAM → Confirmation

└ " Workflow [name] corrigé et réactivé"

Temps de build : 2-3h

Gain : zéro intervention manuelle sur les erreurs courantes`

```
    }
  }
]
},

// _____
// MODULE 8 – CHECKLIST PROJET PAR TYPE
// _____
{
```

```

id: "checklists",
icon: "✅",
label: "Checklists Projets",
color: "#4ADE80",
accent: "#16A34A",
chapters: [
  {
    title: "8.1 – Checklist TUNNEL (avant chaque mise en prod)",
    content: null,
    checklist: [
      "[ ] Service local tourne correctement (curl localhost:PORT → 200)",
      "[ ] cloudflared installé et à jour (cloudflared --version)",
      "[ ] Authentification Cloudflare valide (cloudflared tunnel list)",
      "[ ] Fichier config.yml correctement formaté (yaml lint)",
      "[ ] Route DNS créée dans Cloudflare Dashboard (CNAME visible)",
      "[ ] Test depuis l'extérieur : curl https://service.mondomaine.com",
      "[ ] Service systemd actif : systemctl is-active cloudflared",
      "[ ] Test de reboot : reboot + vérification que le tunnel revient",
      "[ ] Logs sans erreurs : journalctl -u cloudflared --since '5 min ago'",
      "[ ] HTTPS valide (pas d'avertissement SSL dans le navigateur)"
    ]
  },
  {
    title: "8.2 – Checklist SITE WEB (avant livraison)",
    content: null,
    checklist: [
      "[ ] Build production sans erreurs (npm run build)",
      "[ ] Lighthouse > 90 sur Performance, Accessibilité, SEO, Best Practice",
      "[ ] Test mobile : iPhone SE (375px) et Pixel 5 (393px)",
      "[ ] Test tablette : 768px",
      "[ ] Tous les liens fonctionnent (pas de 404)",
      "[ ] Formulaires testés (soumission + validation + messages d'erreur)",
      "[ ] Images : format WebP, taille < 200KB, alt text présent",
      "[ ] Favicon configuré pour tous les appareils",
      "[ ] Meta title < 60 caractères",
      "[ ] Meta description < 160 caractères",
      "[ ] Open Graph configuré (partage réseaux sociaux)",
      "[ ] HTTPS activé sur le domaine",
      "[ ] Sitemap.xml généré et soumis à Google Search Console",
      "[ ] Analytics configuré (si requis)",
      "[ ] Variables d'environnement en production ≠ développement"
    ]
  },
  {
    title: "8.3 – Checklist WORKFLOW N8N (avant activation)",
    content: null,
    checklist: [

```

```

    "[ ] Test avec données mockées (pas de vraies données en dev)",
    "[ ] Tous les nœuds testés individuellement",
    "[ ] Cas d'erreur définis (que se passe-t-il si l'API Claude est down ?",
    "[ ] Credentials configurés dans n8n UI (pas hardcodés dans le JSON)",
    "[ ] Webhook URL testée depuis l'extérieur",
    "[ ] Limits configurées (max iterations, timeout par nœud)",
    "[ ] Workflow d'erreur configuré (Error Trigger → Telegram/Email)",
    "[ ] Logs activés pour les nœuds critiques",
    "[ ] Prompt caching activé si system prompt identique (économie coûts)",
    "[ ] Test en production avec un cas réel supervisé",
    "[ ] Documentation du workflow dans CLAUDE.md du projet"
  ]
},
{
  title: "8.4 – Checklist SETUP INITIAL (à faire une fois)",
  content: null,
  checklist: [
    "[ ] Node.js 20+ installé",
    "[ ] Claude Code installé (npm install -g @anthropic-ai/claude-code)",
    "[ ] claude doctor sans erreur",
    "[ ] ~/.claude/CLAUDE.md créé et personnalisé",
    "[ ] ~/.claude/settings.json créé avec permissions",
    "[ ] ~/.claude/hooks/pre-tool-use.sh créé et rendu exécutable (chmod +x",
    "[ ] Structure skills créée (mkdir -p ~/.claude/skills/...)",
    "[ ] Skills essentiels copiés : code-review, n8n-workflow, tunnel, site",
    "[ ] Variables d'env configurées dans ~/.bashrc (GITHUB_TOKEN, etc.)",
    "[ ] MCP GitHub installé et testé (claude mcp test github)",
    "[ ] MCP Filesystem installé et testé",
    "[ ] Premier projet test créé avec CLAUDE.md et .mcp.json",
    "[ ] Settings claude.ai configurés (Memory, Search, Artifacts activés)",
    "[ ] 2-3 Projects créés sur claude.ai avec instructions"
  ]
}
],
},

// _____
// MODULE 9 – DÉPANNAGE & ERREURS FRÉQUENTES
// _____
{
  id: "debug",
  icon: "🔧",
  label: "Dépannage",
  color: "#FCD34D",
  accent: "#D97706",
  chapters: [
    {

```

```

title: "9.1 – Erreurs les plus fréquentes et solutions",
content: null,
table: {
  headers: ["Erreur", "Cause probable", "Solution"],
  rows: [
    ["Claude Code ne démarre pas", "Node.js < 18 ou pas dans PATH", "node
    ["MCP server not found", "npx non disponible ou version cache", "npx
    ["Variables d'env non reconnues par Claude Code", "Export dans .bashr
    ["CLAUDE.md ignoré après 20 messages", "Context rot – trop de règles"
    ["Tunnel Cloudflare 502", "Service local arrêté ou mauvais port", "cu
    ["Rate limit Claude API dans n8n", "Trop de requêtes simultanées", "A
    ["Skill ne se déclenche pas", "Description du skill trop vague", "Ren
    ["Hook non exécuté", "Fichier non exécutable", "chmod +x ~/.claude/ho
  ]
}
},
{
  title: "9.2 – Commandes de diagnostic complètes",
  content: null,
  code: {
    lang: "bash",
    label: "Toolkit de diagnostic – À exécuter en cas de problème",
    body: `# — Claude Code général —

claude doctor                                # Diagnostic complet
claude --version                             # Vérifier la version
claude config list                           # Lister la configuration

# — MCP —
claude mcp list                              # Tous les serveurs enregistrés
claude mcp test <nom-serveur>                # Tester un serveur
claude mcp list-tools <nom>                  # Outils disponibles d'un serveur
tail -f ~/.claude/logs/mcp.log               # Logs en temps réel

# — CLAUDE.md —
wc -l ~/.claude/CLAUDE.md                    # Compter les lignes (< 200)
claude "Quelles sont tes instructions ?" # Voir ce que Claude a chargé

# — Hooks —
ls -la ~/.claude/hooks/                      # Vérifier les permissions (rwx)
bash -n ~/.claude/hooks/pre-tool-use.sh      # Vérifier la syntaxe

# — Variables d'env —
env | grep -E 'GITHUB|N8N|ANTHROPIC|CLOUDFLARE' # Lister les tokens

# — Tunnel Cloudflare —
cloudflared tunnel list                      # Lister les tunnels
systemctl status cloudflared                 # Status du service

```

```

journalctl -u cloudflared -n 50      # Derniers logs
curl -I https://votre-service.com    # Test HTTP

# — n8n —
curl http://localhost:5678/healthz   # n8n alive ?
docker compose logs n8n --tail=20    # Logs Docker`
    }
  },
  {
    title: "9.3 – Context rot : le problème silencieux",
    content: `Le context rot survient quand Claude « oublie » des règles de v
    code: {
      lang: "bash",
      label: "Solutions anti-context-rot",
      body: `# 1. Compacter la conversation (garde le contexte utile, efface
/compact

# 2. Vérifier combien de tokens sont utilisés
/tokens

# 3. Redémarrer une nouvelle session au début de chaque nouvelle tâche
# (ne pas accumuler les tâches dans une seule session)

# 4. Réduire CLAUDE.md à < 150 lignes (réalité : 100 est mieux)
# Les règles importantes sur les 50 premières lignes

# 5. Utiliser les Hooks pour les règles critiques (100% fiable)
# CLAUDE.md pour les préférences de style (70% fiable)

# 6. Référencer des fichiers plutôt qu'inline :
# Dans CLAUDE.md :
# ## Sécurité
# Lire docs/security-rules.md pour les règles détaillées.
# (Claude lit le fichier seulement quand c'est pertinent)`
    }
  }
]
}
];

// =====
// COMPOSANTS UI
// =====

const getTypeStyle = (type) => ({
  warning: { bg: "#2d1b0e", border: "#92400e", icon: "⚠️" },
  code: { bg: "#0d1117", border: "#21262d", icon: null },

```

```

    table: { bg: "transparent", border: "transparent", icon: null },
    checklist: { bg: "#0f1f0f", border: "#166534", icon: null },
    info: { bg: "#0f172a", border: "#1e3a5f", icon: "i" },
  }[type] || { bg: "#0f172a", border: "#1e293b", icon: null });

```

```

function ContentRenderer({ chapter }) {
  return (
    <div>
      {chapter.content && (
        <p style={{ color: "#94a3b8", lineHeight: 1.7, fontSize: 14, marginBottom: 16 }}>
          {chapter.content}
        </p>
      )}
      {chapter.warning && (
        <div style={{ background: "#2d1b0e", border: "1px solid #92400e", borderR: 10px, padding: 10px }}>
          <div style={{ color: "#fbbf24", fontSize: 13, lineHeight: 1.7, whiteSpace: nowrap }}>
            <span style={{ fontWeight: 700 }}>⚠️ IMPORTANT – </span>{chapter.warning}
          </div>
        </div>
      )}
      {chapter.table && (
        <div style={{ overflowX: "auto", marginBottom: 16 }}>
          <table style={{ width: "100%", borderCollapse: "collapse", fontSize: 13 }}>
            <thead>
              <tr>{chapter.table.headers.map((h, i) => (
                <th key={i} style={{ background: "#1e293b", color: "#64748b", padding: 5px 10px }}>
                  {h}
                </th>
              ))}</tr>
            </thead>
            <tbody>{chapter.table.rows.map((row, ri) => (
              <tr key={ri}>{row.map((cell, ci) => (
                <td key={ci} style={{ padding: "9px 12px", color: ci === 0 ? "#e2e3e5" : "#94a3b8" }}>
                  {cell}
                </td>
              ))}</tr>
            ))}</tbody>
          </table>
        </div>
      )}
      {chapter.code && (
        <div style={{ marginBottom: 16 }}>
          {chapter.code.label && (
            <div style={{ fontSize: 11, color: "#475569", marginBottom: 6, fontFamily: "monospace" }}>
              {chapter.code.label}
            </div>
          )}
          <pre style={{ background: "#0d1117", border: "1px solid #21262d", borderR: 10px, padding: 10px }}>
            {chapter.code.body}
          </pre>
        </div>
      )}
    </div>
  );
}

```



```

    })
    {chapter.checklist && (
      <div style={{ background: "#080f0e", border: "1px solid #14532d", borderR
        {chapter.checklist.map((item, i) => {
          const done = item.startsWith("✅");
          return (
            <div key={i} style={{ display: "flex", gap: 10, padding: "5px 0", a
              <span style={{ fontSize: 14, flexShrink: 0, marginTop: 1 }}>{done
                <span style={{ color: done ? "#4ade80" : "#94a3b8", fontSize: 13,
                  {item.replace(/^✅ /, "")}
                </span>
              </div>
            );
          )}}
        </div>
      )}
    </div>
  );
}

```

```

function Chapter({ chapter, color }) {
  const [open, setOpen] = useState(false);
  return (
    <div style={{ marginBottom: 8, border: "1px solid #1e293b", borderRadius: 10,
      <button onClick={() => setOpen(!open)} style={{ width: "100%", background:
        <span style={{ fontSize: 14, fontWeight: 600, color: "#e2e8f0", lineHeigh
        <span style={{ color, fontSize: 18, flexShrink: 0, transition: "transform
      </button>
      {open && (
        <div style={{ padding: "2px 16px 16px", borderTop: "1px solid #1e293b" }}
          <ContentRenderer chapter={chapter} />
        </div>
      )}
    </div>
  );
}

```

```

// =====
// APP PRINCIPALE
// =====

```

```

export default function App() {
  const [activeId, setActiveId] = useState("overview");
  const [search, setSearch] = useState("");
  const module = MODULES.find(m => m.id === activeId);

  const filtered = search.trim()

```

```

? MODULES.map(m => ({
  ...m,
  chapters: m.chapters.filter(c =>
    c.title.toLowerCase().includes(search.toLowerCase()) ||
    (c.content || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.code?.body || "").toLowerCase().includes(search.toLowerCase())
  )
})).filter(m => m.chapters.length > 0)
: null;

return (
  <div style={{ minHeight: "100vh", background: "#020817", color: "#e2e8f0", fo

    /* Header fixe */
    <div style={{ position: "sticky", top: 0, zIndex: 100, background: "#020817"
      <div>
        <div style={{ fontSize: 11, fontWeight: 700, color: "#475569", letterSpacing
          <div style={{ fontSize: 17, fontWeight: 800, color: "#f1f5f9", lineHeight
        </div>
      <input
        value={search}
        onChange={e => setSearch(e.target.value)}
        placeholder="🔍 Rechercher dans le guide..."
        style={{ marginLeft: "auto", background: "#0f172a", border: "1px solid
      />
    </div>

    <div style={{ display: "flex", minHeight: "calc(100vh - 57px)" }}>

      /* Sidebar navigation */
      <div style={{ width: 220, flexShrink: 0, borderRight: "1px solid #0f172a"
        {MODULES.map(m => (
          <button
            key={m.id}
            onClick={() => { setActiveId(m.id); setSearch(""); }}
            style={{
              width: "100%", background: activeId === m.id ? `${m.color}15` : "
              border: `1px solid ${activeId === m.id ? m.color + "40" : "transp
              borderRadius: 8, padding: "10px 12px", textAlign: "left", cursor:
              marginBottom: 4, display: "flex", alignItems: "center", gap: 10,
            }}
          >
            <span style={{ fontSize: 16 }}>{m.icon}</span>
            <div>
              <div style={{ fontSize: 12, fontWeight: 600, color: activeId ===
            </div>
          </button>

```

```

    )))
  </div>

  { /* Contenu principal */ }
  <div style={{ flex: 1, padding: "24px 28px", overflowY: "auto", maxWidth:
    {search.trim() ? (
      <>
        <div style={{ marginBottom: 20, color: "#64748b", fontSize: 13 }}>
          Résultats pour « <span style={{ color: "#e2e8f0" }}>{search}</spa
        </div>
        {filtered.map(m => (
          <div key={m.id} style={{ marginBottom: 28 }}>
            <div style={{ display: "flex", alignItems: "center", gap: 8, ma
              <span style={{ fontSize: 18 }}>{m.icon}</span>
              <span style={{ fontSize: 13, fontWeight: 700, color: m.color
            </div>
            {m.chapters.map((c, i) => <Chapter key={i} chapter={c} color={m
          </div>
        ))}
        {filtered.length === 0 && (
          <div style={{ color: "#475569", fontSize: 14 }}>Aucun résultat po
        )}
      </>
    ) : module ? (
      <>
        { /* Header du module */ }
        <div style={{ background: `${module.color}10`, border: `1px solid $
          <span style={{ fontSize: 36 }}>{module.icon}</span>
          <div>
            <div style={{ fontSize: 11, fontWeight: 700, color: module.col
              <div style={{ fontSize: 22, fontWeight: 800, color: "#f8fafc" }
                <div style={{ fontSize: 12, color: "#64748b", marginTop: 2 }}>
                  {module.chapters.length} section{module.chapters.length > 1 ?
                </div>
              </div>
            </div>
          </div>
        </div>

        { /* Chapters */ }
        {module.chapters.map((c, i) => <Chapter key={i} chapter={c} color={

        { /* Navigation bas de page */ }
        <div style={{ display: "flex", justifyContent: "space-between", mar
          {(() => {
            const idx = MODULES.findIndex(m => m.id === activeId);
            const prev = MODULES[idx - 1];
            const next = MODULES[idx + 1];
            return (

```

```

    <>
      {prev ? (
        <button onClick={() => setActiveId(prev.id)} style={{ bac
          ← {prev.icon} {prev.label}
        </button>
      ) : <span />}
      {next && (
        <button onClick={() => setActiveId(next.id)} style={{ bac
          {next.icon} {next.label} →
        </button>
      )}
    </>
  );
  })()}
</div>
</>
) : null}
</div>
</div>
</div>
);
}

```